

TEMA N° 2



TEMA 2 USO INCORRECTO DE LA IA EN LA INFORMÁTICA



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Identificar y mitigar los riesgos críticos** asociados al uso desmedido de la Inteligencia Artificial generativa en el desarrollo de software, comprendiendo cómo afecta el pensamiento lógico y técnico.
2. **Evaluar la seguridad y originalidad del código** asistido por herramientas de IA, detectando posibles vulnerabilidades, problemas de derechos de autor y malas prácticas comunes.
3. **Desarrollar una autonomía metodológica sólida** como Técnico en Sistemas Informáticos, utilizando la IA de forma ética y equilibrada para resolver problemas reales en el entorno informático de nuestra comunidad.



PRÁCTICA



PREGUNTA DETONANTE

Si una Inteligencia Artificial puede escribir un código de programación completo en un segundo, ¿por qué los negocios informáticos de la población de Pailón siguen necesitando contratar a un Técnico para desarrollar y mantener sus sistemas?

EL CASO DEL SISTEMA DE COBRANZAS EN EL BARRIO CENTRAL FÁTIMA

Imagine la siguiente situación real en nuestro municipio de Pailón: el propietario de una gran tienda de comercio formal ubicada en el **Barrio Central Fátima**, a pocos pasos de la iglesia principal, decide automatizar el control de sus ventas y créditos de mercadería. Para ahorrar costos, en lugar de contratar una consultoría de software completa, le pide a un estudiante del CEA que actúe como programador independiente.

El estudiante, entusiasmado por el nuevo módulo educativo equipado que tenemos en el **Barrio Saó**, decide avanzar a toda velocidad. En lugar de sentarse a diseñar la base de datos con papel y lápiz, abre una herramienta de Inteligencia Artificial generativa y le escribe el siguiente mensaje: *"Hazme un script en Python para controlar clientes, deudas y que envíe alertas de pago automáticamente"*. La IA le devuelve un código de más de doscientas líneas en un par de segundos. El estudiante, maravillado por la rapidez, copia el código completo, lo pega en su editor y lo ejecuta. En la pantalla aparece una interfaz limpia y funcional.

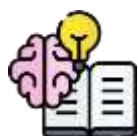
Durante la primera semana en el Barrio Central Fátima, el sistema parece un éxito rotundo. Las ventas se registran y los clientes se guardan en la base de datos. Sin embargo, el día sábado, cuando la tienda se llena de compradores de las comunidades vecinas de Pailón, el sistema se congela por completo al procesar el pago número cincuenta. El software empieza a duplicar los saldos deudores de varios clientes fijos y, para empeorar las cosas, el sistema expone en la red local los números de carnet y datos privados de los clientes.

El comerciante, desesperado y viendo que se genera un caos en su negocio, le pide al estudiante que lo arregle de inmediato. El estudiante abre el código fuente generado por la IA, pero se da



cuenta de una amarga realidad: **no entiende qué hace la mitad de las funciones**, no sabe cómo se estructuraron los bucles de control y no tiene idea de dónde proviene el fallo técnico. Al intentar pedirle ayuda nuevamente a la IA con el mensaje "*Mi código falló, arréglalo*", la herramienta le devuelve una solución completamente diferente que desconfigura aún más el sistema, borrando los registros de ventas de toda la mañana.

Este escenario real nos demuestra que el uso ciego de la tecnología emergente puede paralizar la economía local de nuestros barrios si no se cuenta con las bases lógicas necesarias.



TEORÍA



2.1. EL RIESGO DE LA DEPENDENCIA EXCESIVA EN HERRAMIENTAS GENERATIVAS

El auge de los Modelos de Lenguaje Grandes (LLM) ha transformado la informática de una manera sin precedentes. Como futuro Técnico, usted tiene acceso a asistentes virtuales capaces de generar funciones, estructurar bases de datos y maquetar interfaces en lenguajes como HTML, JavaScript o Python de forma casi instantánea. No obstante, esta asombrosa velocidad esconde una trampa metodológica peligrosa: la **falsa sensación de competencia**.

Cuando un programador principiante se acostumbra a delegar la escritura del 100% de su código a una IA, deja de realizar el esfuerzo cognitivo fundamental para asimilar las estructuras de control. La dependencia excesiva convierte al profesional en un simple "copiador y pegador" de texto. El riesgo radica en que las herramientas generativas no "piensan" en el sentido humano; ellas predicen secuencias de palabras basándose en patrones estadísticos de datos históricos. Por lo tanto, si la herramienta introduce un error lógico sutil que encaja en el patrón estadístico, el desarrollador dependiente será totalmente incapaz de percibirlo, poniendo en riesgo la estabilidad del software institucional.

2.2. ATROFIA DE LA LÓGICA COMPUTACIONAL EN ESTUDIANTES

La programación de computadoras no consiste en escribir texto rápidamente; consiste en **diseñar soluciones lógicas estructuradas para resolver problemas humanos**. La lógica



computacional se desarrolla mediante la práctica constante, la prueba y el error, y el análisis paso a paso de los flujos de datos.

Cuando saltamos el proceso de razonamiento y dejamos que la IA redacte los algoritmos por nosotros, ocurre un fenómeno conocido como la atrofia del pensamiento abstracto. Si un estudiante del CEA no practica la construcción manual de estructuras condicionales (if-else), bucles (while, for) u operaciones recursivas, su cerebro pierde la capacidad de descomponer problemas complejos en subproblemas más pequeños.



Para entender la diferencia entre un código analizado y uno copiado a ciegas, observemos cómo se procesaría de forma incorrecta el cálculo de un descuento por volumen de compra para los comerciantes del **Barrio 24 de Septiembre** en la estación de trenes de Pailón. Si confiamos ciegamente en una IA, podríamos obtener un bucle mal estructurado como este:

Python

```
# Código generado sin revisión para calcular descuentos de clientes en el Barrio
24 de Septiembre
def calcular_descuentos_tienda(lista_compras):
    total = 0
    # Iteramos sobre los productos de la compra actual
    for i in range(len(lista_compras)):
        # La IA asume erróneamente que el precio ya incluye impuestos locales
        precio_producto = lista_compras[i]['precio']
        cantidad = lista_compras[i]['
```




```
cantidad']           # Un error grave de Lógica: si La cantidad es mayor a
10, reduce el precio a cero           if cantidad > 10:           total +=
(precio_producto * 0) # Error: Debería aplicar un porcentaje, no anular el valor
else:           total += (precio_producto * cantidad)           return
total
```

Un Técnico que comprende la lógica computacional se daría cuenta inmediatamente de que la condición `precio_producto * 0` destruye las ganancias del negocio, un error que una IA puede escribir con total naturalidad formal.

2.3. PLAGIO DE CÓDIGO Y VIOLACIONES DE DERECHOS DE AUTOR

Las herramientas de Inteligencia Artificial han sido entrenadas utilizando miles de millones de líneas de código fuente alojadas en repositorios públicos de internet, como GitHub. Gran parte de este código está protegido por licencias de software específicas (como la Licencia Pública General GNU, licencias Apache o licencias Creative Commons) que exigen dar crédito al autor original o mantener el código abierto.

Al usar código generado por IA, existe una alta probabilidad de cometer **plagio involuntario** o infringir derechos de autor. Las herramientas generativas suelen reproducir bloques de código idénticos a proyectos registrados sin incluir las licencias obligatorias. Si usted, como Técnico en Sistemas Informáticos, implementa un software para una entidad financiera en el Barrio Central Fátima o para una institución pública en el **Barrio 27 de Mayo**, y ese software contiene fragmentos con derechos reservados extraídos por una IA, la organización podría enfrentarse a demandas legales severas por uso no autorizado de propiedad intelectual. La ética profesional exige verificar siempre la originalidad y los términos de uso del código que implementamos.

2.4. GENERACIÓN E INYECCIÓN DE CÓDIGO CON VULNERABILIDADES DE SEGURIDAD

La IA no evalúa si el código que genera cumple con los estándares internacionales de ciberseguridad, como las guías de OWASP (Open Web Application Security Project). Debido a que recopila información de foros antiguos y ejemplos de internet que a veces están desactualizados, con frecuencia entrega códigos que contienen fallos de seguridad graves.

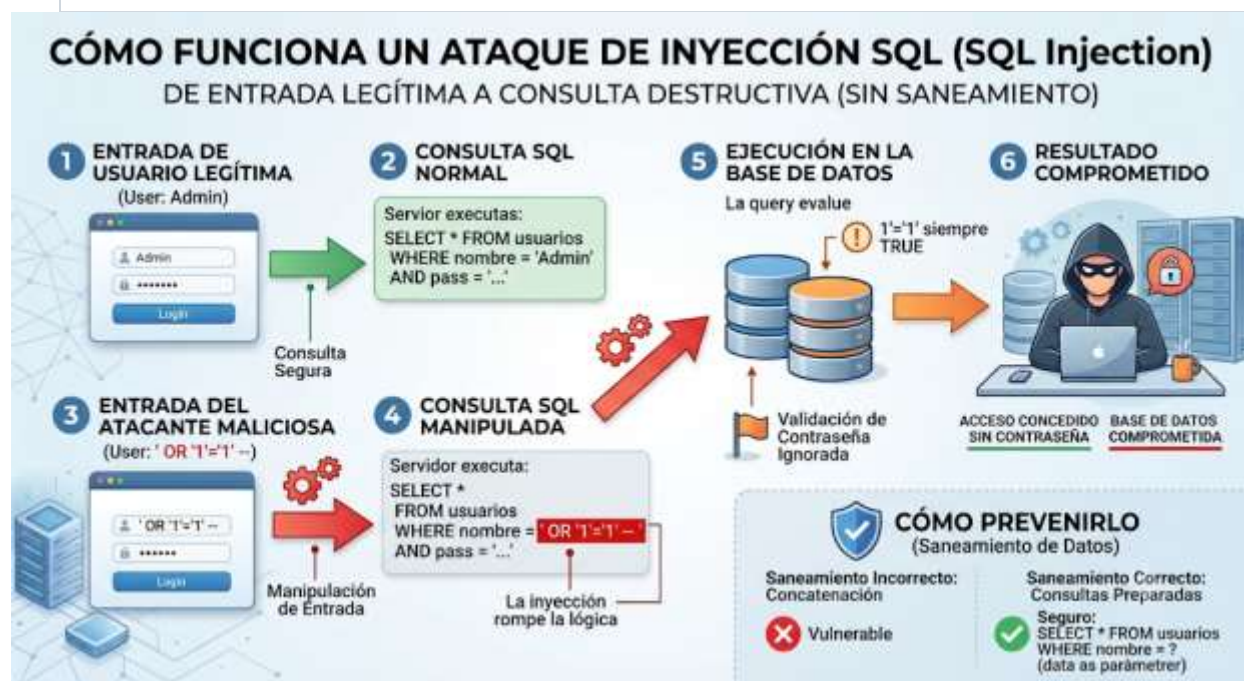
Uno de los ataques más peligrosos en informática es la **Inyección SQL**, la cual ocurre cuando un sistema permite que un usuario malintencionado introduzca comandos directamente en la base



de datos a través de un formulario web. Veamos un ejemplo práctico de un código vulnerable creado por una IA para un sistema de reservas del coliseo municipal en el Barrio 27 de Mayo:

Python

```
import sqlite3
# Código inseguro generado por IA para validar usuarios del coliseo en el Barrio
27 de Mayo
def buscar_usuario_formulario(nombre_usuario):
    conexion = sqlite3.connect("usuarios_coliseo.db")
    cursor = conexion.cursor()
    # ERROR DE SEGURIDAD CRÍTICO: Concatenación directa de variables en la consulta SQL
    # Esto permite que un atacante inyecte código malicioso en la base de datos
    consulta = "SELECT * FROM usuarios WHERE nombre = '" + nombre_usuario + "'"
    cursor.execute(consulta)
    resultado = cursor.fetchall()
    conexion.close()
    return resultado
```



Si un atacante escribe en el formulario la cadena ' OR '1'='1', la consulta final se convertirá en SELECT * FROM usuarios WHERE nombre = ' OR '1'='1', dándole acceso inmediato a todas las contraseñas guardadas del coliseo. Un Técnico capacitado sabe que debe utilizar consultas preparadas y parametrizadas para evitar que el software local sea hackeado con facilidad.



2.5. EL IMPACTO NEGATIVO EN LA RESOLUCIÓN DE PROBLEMAS (TROUBLESHOOTING)

La habilidad más valiosa de un Técnico no es escribir código sin errores, sino saber **encontrar y reparar el error cuando el sistema deja de funcionar**. Este proceso técnico se denomina *Troubleshooting* o depuración.

Cuando un profesional depende por completo de la IA, el proceso de depuración se rompe. En lugar de leer los logs del servidor local del CEA Herman Ortiz Camargo o analizar las trazas de error de la consola del navegador, el desarrollador se limita a copiar el mensaje de error completo, pegarlo en la IA y esperar que la máquina lo solucione. Esto crea un ciclo destructivo: la IA propone parches aleatorios que solucionan el síntoma del momento pero dañan otras partes de la arquitectura del software, haciendo que el código se vuelva incomprensible, inestable y propenso a colapsar ante cualquier cambio de requerimientos.

2.6. SESGO ALGORÍTMICO Y ALUCINACIONES EN EL DESARROLLO DE SOFTWARE LOCAL

Las "alucinaciones" son un fenómeno técnico documentado donde la Inteligencia Artificial inventa información con absoluta seguridad formal. En informática, esto se traduce en la creación de funciones que no existen, invocación de librerías de programación inexistentes o recomendación de configuraciones de red obsoletas.

Para los proyectos tecnológicos en Pailón, como los sistemas de riego o redes del **Barrio Residencial Norte**, esto puede causar pérdidas de tiempo críticas. Un programador puede pasar horas intentando instalar un paquete informático que la IA inventó. Asimismo, los modelos sufren de sesgos geográficos y sociales: la IA suele asumir que todos los entornos tienen conexiones de internet de alta velocidad con fibra óptica o servidores en la nube de última generación, ignorando la realidad de muchas redes locales o conexiones intermitentes en las provincias chiquitana.



2.7. PÉRDIDA DE LA PRIVACIDAD DE DATOS INSTITUCIONALES AL ENTRENAR MODELOS PÚBLICOS

Cada vez que un usuario interactúa con una plataforma de IA comercial gratuita, los datos enviados en el mensaje de chat (prompts) son almacenados y procesados por las empresas tecnológicas extranjeras para seguir entrenando a sus modelos.



Si un Técnico del CEA Herman Ortiz Camargo toma la base de datos Excel con los pagos de mensualidades de los estudiantes, o las planillas del personal de la Escuela de Fútbol Unidas del municipio, y las sube a una IA para que realice un gráfico estadístico, está incurriendo en una **violación grave de la privacidad de datos**. Esos nombres, números telefónicos y estados financieros quedan flotando en la red corporativa de la IA, pudiendo ser expuestos posteriormente a otros usuarios mediante ingeniería inversa.

2.8. BUENAS PRÁCTICAS Y EL ROL DEL TÉCNICO FRENTE AL USO ÉTICO DE LA IA

La Inteligencia Artificial no debe ser vista como un enemigo, sino como un **asistente de copiloto altamente eficiente**. El verdadero secreto radica en mantener siempre el control del volante tecnológico. Las buenas prácticas exigen que el Técnico escriba primero su estructura lógica, use la IA únicamente para optimizar partes específicas del código o generar documentación explicativa, y revise minuciosamente cada línea generada antes de ponerla en producción. Su



firma profesional como Técnico avala la seguridad y calidad del software; la máquina no asume responsabilidades éticas ni técnicas.

CIERRE TEÓRICO OBLIGATORIO

❑ Mitos vs. Realidades

Mito Corporativo	Realidad Técnica Comprobada
"La Inteligencia Artificial ya no comete errores de programación, por lo que probar el software manualmente ya es obsoleto."	La IA hereda errores humanos de los foros de internet, generando software con fallas de seguridad y lógica que solo un Técnico puede detectar.
"Cualquier persona sin conocimientos de informática puede crear sistemas profesionales usando herramientas generativas."	Las personas sin bases lógicas crean sistemas deficientes, inseguros y que colapsan ante el primer problema técnico en el entorno productivo real.

❑ Glosario de Términos

1. **Alucinación de IA:** Fenómeno técnico en el cual un modelo de lenguaje generativo inventa datos, librerías o respuestas incorrectas, presentándolas como verdaderas.
2. **Inyección SQL:** Vulnerabilidad de seguridad crítica donde un atacante logra manipular las consultas de la base de datos introduciendo código malicioso a través de las entradas de usuario.
3. **Troubleshooting:** Proceso metodológico y ordenado de identificación, análisis y resolución de problemas técnicos dentro de un sistema de hardware o software.
4. **Saneamiento de Datos:** Técnica de programación que consiste en limpiar y validar los datos introducidos por un usuario antes de procesarlos, evitando ataques informáticos.
5. **Licencia de Software:** Marco legal que define los derechos, restricciones y obligaciones del usuario respecto al uso, modificación y distribución de un código fuente.



□ Ponte a Prueba

1. **Cuando un programador copia y pega código de una IA de forma repetitiva sin estudiar sus componentes, ¿qué capacidad técnica principal corre el riesgo de atrofiarse?**

1. A) La velocidad de escritura en el teclado de la computadora.
2. B) La capacidad de abstracción y desarrollo de la lógica computacional propia.
3. C) El acceso a servicios de almacenamiento en la nube del CEA.

2. **¿Por qué representa un peligro de seguridad utilizar directamente scripts de base de datos creados por herramientas de IA?**

1. A) Porque los scripts consumen demasiada memoria RAM en los servidores locales.
2. B) Porque las herramientas omiten frecuentemente el saneamiento de variables, introduciendo fallos como la inyección SQL.
3. C) Porque la base de datos solo funciona en sistemas operativos de pago.

3. **Si usted introduce datos privados de los clientes de una empresa de Pailón en un chat público de IA, ¿qué regla informática está vulnerando?**

1. A) La optimización del rendimiento del software web.
2. B) La confidencialidad y privacidad de la información institucional.
3. C) La compatibilidad con navegadores antiguos de internet.



VALORACIÓN



REFLEXIÓN PROFUNDA SOBRE EL IMPACTO TECNOLÓGICO EN PAILÓN

El uso irresponsable de la Inteligencia Artificial no solo acarrea problemas técnicos individuales, sino que genera un fuerte impacto ético y social en nuestra comunidad. En el ámbito de la **seguridad y la privacidad**, delegar la administración de sistemas locales de salud, educación o



comercio a herramientas automatizadas externas centraliza el control de nuestra información en servidores extranjeros. Esto despoja a nuestras comunidades provinciales de la soberanía tecnológica necesaria para autogobernarse digitalmente.

Desde el punto de vista **ético y social**, el abuso de la IA fomenta una cultura del mínimo esfuerzo entre los nuevos profesionales de la región. Si los profesionales informáticos locales pierden la habilidad de resolver problemas complejos de manera independiente, la región de Pailón y todo el departamento de Santa Cruz se volverán crónicamente dependientes de soporte técnico especializado externo, frenando la innovación productiva de las pequeñas empresas de nuestros barrios.

Asimismo, existe un impacto indirecto relacionado con el **e-waste (basura electrónica)**. El entrenamiento y mantenimiento de los colosales centros de datos de las IA consumen millones de litros de agua dulce y generan una huella de carbono masiva. Cuando un sistema desarrollado de forma ineficiente por una IA falla constantemente y satura el hardware de los servidores de un centro educativo como el nuestro, obliga a las instituciones a desechar computadoras prematuramente por obsolescencia del software, incrementando los residuos tecnológicos que dañan el medioambiente de las zonas rurales de Santa Cruz.



PRODUCCIÓN



RETO FINAL: AUDITORÍA Y REFACTORIZACIÓN MANUAL DE CÓDIGO INSEGURO

Objetivo del Laboratorio: Actuar como un Técnico encargado de auditar, detectar y reparar los fallos de seguridad y lógica en un sistema informático que fue generado de forma incorrecta por una IA para el control de inventarios de herramientas en el taller de informática del **Barrio Saó**.

Paso 1: Análisis del código defectuoso

En la computadora de desarrollo instalada en el nuevo módulo del CEA Herman Ortiz Camargo, abra su editor de texto (como Visual Studio Code) y copie el siguiente script defectuoso que un programador descuidado obtuvo de una IA pública:



Python

```
# SCRIPT ORIGINAL GENERADO POR IA - CONTROL DE HERRAMIENTAS EN BARRIO SAÓdef
registrar_prestamo_herramientas():    # Simulamos los datos recibidos del
formulario del taller del CEA    usuario_id = input("
Ingrese el ID del estudiante del CEA: ")    nombre_herramienta = input("
Ingrese el nombre de la herramienta del taller: ")    cantidad = input("
Ingrese la cantidad solicitada para la práctica: ")    # FALLO DE LÓGICA 1:
La IA guarda la cantidad directamente como texto sin validar si es un número
válido    print("
Buscando herramienta en el inventario de la base de datos...")    # FALLO DE
SEGURIDAD 2: Inyección SQL potencial al concatenar variables sin sanear
consulta_base_datos = "
INSERT INTO prestamos VALUES ('" + usuario_id + "', '" + nombre_herramienta +
"', " + cantidad + " )"    print("
Consulta SQL que se ejecutará en el servidor local:")
print(consulta_base_datos)    # FALLO DE LÓGICA 3: No se verifica si la
cantidad es negativa o cero antes de confirmar el préstamo    print("¡Préstamo
registrado con éxito mediante IA sin verificar stock real!")
# Ejecución del script defectuoso
registrar_prestamo_herramientas()
```

Paso 2: Ejecución e Inducción del Error

1. Ejecute el script en la terminal utilizando el comando `python nombre_archivo.py`.
2. En el campo de cantidad, introduzca un valor negativo como -5. Note cómo el script permite avanzar sin lanzar ninguna advertencia, lo que arruinaría el inventario físico del taller informático del CEA.
3. Vuelva a ejecutar el script y, en el campo `usuario_id`, introduzca la cadena de inyección SQL: `' O '1'='1`. Observe cómo la consulta se rompe por completo, dejando expuesta la sintaxis interna del motor de base de datos.

Paso 3: Refactorización y Reparación Técnica (La Solución)

Cree un nuevo archivo llamado `prestamo_seguro.py` y programe la solución correcta aplicando las bases de la lógica computacional y la ciberseguridad defensiva. Su código final debe verse de la siguiente manera:



Python

```
# SCRIPT CORREGIDO Y AUDITADO POR EL TÉCNICO EN SISTEMAS INFORMÁTICOS
import sqlite3
def registrar_prestamo_seguro():    print("
--- SISTEMA DE CONTROL DE TALLER - CEA HERMAN ORTIZ CAMARGO -
--")    usuario_id = input("
Ingrese el ID del estudiante del CEA: ").strip()    nombre_herramienta = input("
Ingrese el nombre de la herramienta del taller: ").strip()    # VALIDACIÓN
LÓGICA: Aseguramos que la cantidad sea un número entero válido    try:
cantidad = int(input("
Ingrese la cantidad solicitada para la práctica: "))    except ValueError:
print("[X] ERROR TÉCNICO: La cantidad introducida debe ser un número entero
válido.")    return    # CONTROL DE REGLAS DE NEGOCIO: Evitamos valores
negativos o ceros    if cantidad <= 0:    print("[X] ERROR LÓGICA: No se
pueden realizar préstamos de cantidades menores o iguales a cero.")
return    # CONEXIÓN SEGURA Y PREVENCIÓN DE INYECCIÓN SQL
print("\n[+] Conectando de forma segura a la base de datos local del Barrio
Saó...")    # Usamos marcadores de posición (?) en lugar de concatenación
directa de texto    consulta_segura = "
INSERT INTO prestamos (estudiante_id, herramienta, cantidad_prestada) VALUES (?,
?, ?)"    print("[+] Consulta estructurada y protegida lista para ser
procesada:")    print(f"
Comando base: {
consulta_segura}
")    print(f"
Parámetros limpios enviados: ({
usuario_id}
, {
nombre_herramienta}
, {
cantidad}
)")    print("[+] ¡El registro se completó de manera segura bajo la supervisión
del Técnico!")
# Ejecución del código refactorizado
registrar_prestamo_seguro()
```

Al terminar este laboratorio, guarde sus dos archivos en el servidor local del CEA como evidencia del desarrollo de su lógica profesional independiente frente al automatismo de las herramientas informáticas.

RECURSOS ADICIONALES

1. **OWASP Top 10 Reference Guide:** Guía oficial internacional indispensable para estudiar las diez vulnerabilidades de seguridad de software más críticas del mundo y aprender cómo bloquearlas manualmente en los sistemas que desarrollamos.



2. **GitHub Copilot Trust Center Documentation:** Manual oficial que detalla los aspectos éticos, los derechos de autor de los repositorios de software libre y las políticas de privacidad de datos al utilizar asistentes de código en la informática contemporánea.
3. **Python Software Foundation - Database API Specification:** Documentación técnica oficial que detalla los métodos estándar para realizar consultas parametrizadas seguras a bases de datos, garantizando la inmunidad de las aplicaciones frente a ataques de inyección de código.